

ASSIGNMENT-3.2

Batch-37
2303A52314

Task Description-1

- Progressive Prompting for Calculator Design: Ask the AI to design a simple calculator program by initially providing only the function name. Gradually enhance the prompt by adding comments and usage examples.

Expected Output-1

- Comparison showing improvement in AI-generated calculator logic and structure.

Prompt:-

- 1) A simple calculator class that performs basic arithmetic operations
- 2) Simple calculator class that performs arithmetic and error handling perform square root

Code:-

1)

```
class SimpleCalculator:
    """A simple calculator class that performs basic arithmetic
    operations."""

    def add(self, x, y):
        """Adds two numbers and returns the sum."""
        return x + y

    def subtract(self, x, y):
        """Subtracts the second number from the first and returns the
        difference."""
        return x - y

    def multiply(self, x, y):
```

```

        """Multiplies two numbers and returns the product."""
        return x * y

    def divide(self, x, y):
        """Divides the first number by the second and returns the
quotient.
Handles division by zero by returning an error message.
"""
        if y == 0:
            return "Error! Division by zero is not allowed."
        return x / y

# Example Usage:
calculator = SimpleCalculator()
num1 = 10
num2 = 5
print(f"{num1} + {num2} = {calculator.add(num1, num2)}")
print(f"{num1} - {num2} = {calculator.subtract(num1, num2)}")
print(f"{num1} * {num2} = {calculator.multiply(num1, num2)}")
print(f"{num1} / {num2} = {calculator.divide(num1, num2)}")

# Test division by zero
num3 = 0
print(f"{num1} / {num3} = {calculator.divide(num1, num3)}")

```

2)

```

class SimpleCalculator:
    """A simple calculator class that performs basic arithmetic
operations."""

    def add(self, x, y):
        """Adds two numbers and returns the sum."""
        return x + y

    def subtract(self, x, y):
        """Subtracts the second number from the first and returns the
difference."""
        return x - y

```

```

def multiply(self, x, y):
    """Multiplies two numbers and returns the product."""
    return x * y

def divide(self, x, y):
    """Divides the first number by the second and returns the
quotient.
Handles division by zero by returning an error message.
"""
    if y == 0:
        return "Error! Division by zero is not allowed."
    return x / y

def power(self, x, y):
    """Raises x to the power of y."""
    return x ** y

def modulo(self, x, y):
    """Returns the remainder of x divided by y.
Handles division by zero by returning an error message.
"""
    if y == 0:
        return "Error! Division by zero is not allowed for modulo."
    return x % y

def square_root(self, x):
    """Returns the square root of x.
Handles negative input by returning an error message.
"""
    if x < 0:
        return "Error! Cannot calculate square root of a negative
number."
    return x ** 0.5

# Example Usage:
calculator = SimpleCalculator()
num1 = 10
num2 = 5
print(f"{num1} + {num2} = {calculator.add(num1, num2)}")
print(f"{num1} - {num2} = {calculator.subtract(num1, num2)}")

```

```

print(f"{num1} * {num2} = {calculator.multiply(num1, num2)}")
print(f"{num1} / {num2} = {calculator.divide(num1, num2)}")

# Test division by zero
num3 = 0
print(f"{num1} / {num3} = {calculator.divide(num1, num3)}")

# Example usage of new features
print(f"{num1} to the power of {num2} = {calculator.power(num1, num2)}")
print(f"{num1} modulo {num2} = {calculator.modulo(num1, num2)}")
print(f"Square root of {num1} = {calculator.square_root(num1)}")

# Test modulo by zero
print(f"{num1} modulo {num3} = {calculator.modulo(num1, num3)}")

# Test square root of negative number
num4 = -9
print(f"Square root of {num4} = {calculator.square_root(num4)}")

```

Explain code:-

This code defines a SimpleCalculator class that can perform basic mathematical operations like addition, subtraction, multiplication, division, power, modulo, and square root. Each method handles common error cases, such as division or modulo by zero and finding the square root of a negative number, by returning a clear error message instead of crashing. At the end, the class is tested with different values to show how each operation works and how errors are handled in real situations.

Task Description-2

- Refining Prompts for Sorting Logic: Start with a vague prompt for sorting student marks, then refine it to clearly specify sorting order and constraints.

Expected Output-2

- AI-generated sorting function evolves from ambiguous logic to an accurate and efficient Implementation.

Prompt:-

Write a Python program that asks the user to enter student marks and sorts them in ascending order. Make sure the program handles invalid input and displays the sorted result clearly.

Code:-

```
#Start with a vague prompt for sorting student marks,
#then refine it to clearly specify sorting order and constraints.
def sort_student_marks():
    """Sorts a list of student marks in ascending order.
Usage example:
    sort_student_marks()"""
    print("Welcome to the Student Marks Sorter!")
    marks_input = input("Enter student marks separated by spaces: ")
    try:
        marks = list(map(float, marks_input.split()))
    except ValueError:
        print("Error: Invalid input. Please enter numeric values only.")
        return
    sorted_marks = sorted(marks)
    print("Sorted student marks in ascending order:")
    print(sorted_marks)
if __name__ == '__main__':
    sort_student_marks()
```

Explain code:-

This code defines a function that takes student marks from the user and sorts them in ascending order. It checks for invalid input so the program does not crash if the user enters something other than numbers. When run, it displays a friendly message and prints the sorted marks clearly.

Task Description-3

- Few-Shot Prompting for Prime Number Validation: Provide multiple input-output examples for a function that checks whether a number is prime. Observe how few-shot prompting improves correctness.

Expected Output-3

- Improved prime-checking function with better edge-case handling.

Prompt:-

Write a Python program to check whether a given number is prime.

Code:-

```
#Few-Shot Prompting for Prime Number Validation: Provide multiple input-
output
#examples for a function that checks whether a number is prime. Observe
how few-shot
#prompting improves correctness.
def is_prime():
    """Checks whether a given number is prime.
Usage example:
    is_prime()"""
    print("Welcome to the Prime Number Validator!")
    try:
        num = int(input("Enter a number to check if it is prime: "))
        if num <= 1:
            print(f"{num} is not a prime number.")
            return
        for i in range(2, int(num**0.5) + 1):
            if num % i == 0:
                print(f"{num} is not a prime number.")
                return
        print(f"{num} is a prime number.")
    except ValueError:
        print("Error: Invalid input. Please enter an integer value.")
if __name__ == '__main__':
    is_prime()
```

Explain code:-

This program asks the user to enter a number and checks whether it is a prime number or not. It handles invalid inputs using error handling and avoids unnecessary checks by looping only up to the square root of the number. Finally, it prints a clear message telling whether the number is prime.

Task Description-4

- Prompt-Guided UI Design for Student Grading System: Create a user interface for a student grading system that calculates total marks, percentage, and grade based on user Input.

Expected Output-4

- Well-structured UI code with accurate calculations and clear output display.

Prompt:-

Write a Python program to create a simple student grading system that takes a student's name, roll number, and marks of 5 subjects as input. The program should calculate total marks, percentage, and display the final grade based on the percentage.

Code:-

```
#Prompt-Guided UI Design for Student Grading System: Create a user
interface for a
#student grading system that calculates total marks, percentage, and grade
based on user
#input
def student_grading_system():
    """Calculates total marks, percentage, and grade for a student based
    on input marks.
    Usage example:
    student_grading_system()"""
    print("Welcome to the Student Grading System!")
    try:
        name = input("Enter student name: ")
        roll_number = input("Enter roll number: ")
        marks_input = input("Enter marks for 5 subjects separated by
spaces: ")
        marks = list(map(float, marks_input.split()))
        if len(marks) != 5:
            print("Error: Please enter marks for exactly 5 subjects.")
            return
        total_marks = sum(marks)
        percentage = (total_marks / 500) * 100
        if percentage >= 90:
            grade = 'A'
        elif percentage >= 80:
```

```

        grade = 'B'
    elif percentage >= 70:
        grade = 'C'
    elif percentage >= 60:
        grade = 'D'
    else:
        grade = 'F'
    print(f"\nStudent Name: {name}")
    print(f"Roll Number: {roll_number}")
    print(f"Total Marks: {total_marks}/500")
    print(f"Percentage: {percentage:.2f}%")
    print(f"Grade: {grade}")
except ValueError:
    print("Error: Invalid input. Please enter numeric values for
marks.")
if __name__ == '__main__':
    student_grading_system()

```

Explain code:-

This program takes a student's name, roll number, and marks for five subjects as input and then calculates the total marks and percentage. Based on the percentage, it assigns a grade using simple conditional statements. Finally, it displays all the student details, results, and grade in a clear format.

Task Description-5

- Analyzing Prompt Specificity in Unit Conversion Functions: Improving a Unit Conversion Function (Kilometers to Miles and Miles to Kilometers) Using Clear Instructions.

Expected Output-5

- Analysis of code quality and accuracy differences across multiple prompt variations.

Prompt:-

Write a Python program that allows the user to convert distances between kilometers and miles. The program should ask the user to choose the conversion type, take the distance as input, and display the converted value while handling invalid inputs gracefully.

Code:-

```
#Analyzing Prompt Specificity in Unit Conversion Functions: Improving a
Unit
#Conversion Function (Kilometers to Miles and Miles to Kilometers) Using
Clear
#Instructions.
def unit_conversion():
    """Converts distances between kilometers and miles based on user
input.
Usage example:
    unit_conversion()"""
    print("Welcome to the Unit Conversion Tool!")
    print("Select conversion type:")
    print("1. Kilometers to Miles")
    print("2. Miles to Kilometers")
    conversion_type = input("Enter conversion type (1 or 2): ")
    try:
        if conversion_type == '1':
            km = float(input("Enter distance in kilometers: "))
            miles = km * 0.621371
            print(f"{km} kilometers is equal to {miles:.2f} miles.")
        elif conversion_type == '2':
            miles = float(input("Enter distance in miles: "))
            km = miles / 0.621371
            print(f"{miles} miles is equal to {km:.2f} kilometers.")
        else:
            print("Invalid conversion type selected.")
    except ValueError:
        print("Error: Invalid input. Please enter numeric values for
distance.")
if __name__ == '__main__':
    unit_conversion()
```

Explain code:-

This program lets the user choose whether to convert kilometers to miles or miles to kilometers. It takes the distance as input, performs the appropriate calculation, and displays the result with proper formatting. The code also handles invalid choices and non-numeric inputs to prevent errors and improve user experience.

